

Guía Práctica: concurrencia en PostgreSQL

Objetivo: Observar cómo el Motor de Base de Datos gestiona la concurrencia, los bloqueos y los niveles de aislamiento.

Requisitos: Docker corriendo, pgAdmin abierto con dos pestañas de Query Tool (Sesión A y Sesión B).

Ejercicio 1

Imagina que un cliente está pagando una compra y a su vez alguien le está depositando plata.

- A. **[Sesión A]:** Inicia una transacción y desconta \$500 a la billetera del cliente 1. (No ejecutes **COMMIT** todavía).

```
BEGIN;
```

```
UPDATE billetera_clientes SET saldo = saldo - 500 WHERE  
id_cliente = 1;
```

- B. **[Sesión B]:** Inicia otra transacción e intenta sumar \$1,000 a la billetera de ese *mismo* cliente 1.

```
BEGIN;
```

```
UPDATE billetera_clientes SET saldo = saldo + 1000 WHERE  
id_cliente = 1;
```

- C. **Responde:** ¿Qué sucedió en la pantalla de la Sesión B al ejecutar el UPDATE?
¿Por qué se quedó "cargando"?
- D. **[Sesión A]:** Ejecuta **COMMIT**;
- E. **Responde:** ¿Qué ocurrió inmediatamente en la Sesión B tras el COMMIT de la Sesión A? Ejecuta **COMMIT**; en la Sesión B para finalizar.

Ejercicio 2

Imagina que dos vendedores consultan el stock de un artículo al mismo tiempo, ven que quedan 5 unidades, y ambos clientes quieren comprar esas 5 unidades.

1. **[Sesión A]:** Inicia una transacción y busca el stock del artículo 2, pero exigiendo un bloqueo exclusivo para que nadie más lo toque mientras calculas la venta.

```
BEGIN;
```

```
SELECT stock FROM articulos WHERE id_articulo = 2 FOR UPDATE;
```

2. **[Sesión B]:** El segundo vendedor intenta hacer exactamente lo mismo con el mismo artículo.

```
BEGIN;  
SELECT stock FROM articulos WHERE id_articulo = 2 FOR UPDATE;
```

3. **Responde:** ¿Qué sucede en la Sesión B?
4. **[Sesión A]:** El primer vendedor registra el pedido y actualiza el stock, finalizando la venta.

```
UPDATE articulos SET stock = stock - 5 WHERE id_articulo = 2;  
COMMIT;
```

5. **Responde:** ¿Qué valor de stock ve ahora la Sesión B (que se acaba de destrabar)?
¿Sirvió el `FOR UPDATE` para evitar vender sin stock?
Finaliza ejecutando un `COMMIT`; (o `ROLLBACK`;) en la Sesión B para cerrar la transacción.

Ejercicio 3 - Bloqueos Compartidos (FOR SHARE)

Queremos generar un reporte complejo de los pedidos del cliente 3. Necesitamos asegurarnos de que nadie elimine ni modifique sus datos mientras leemos, pero **sí** queremos permitir que otros vendedores también puedan leer sus datos.

1. **[Sesión A]:** Inicia la transacción y bloquea al cliente 3 en modo compartido.

```
BEGIN;  
SELECT * FROM clientes WHERE id_cliente = 3 FOR SHARE;
```
2. **[Sesión B]:** Otro vendedor también quiere consultar los datos del cliente 3 en modo compartido.

```
BEGIN;  
SELECT * FROM clientes WHERE id_cliente = 3 FOR SHARE;
```
3. **Responde:** ¿La Sesión B se bloqueó o pudo leer los datos inmediatamente? ¿Por qué?
4. **[Sesión A]:** (Opcional) Intenta ejecutar un `UPDATE` cambiando el nombre del cliente
¿Qué ocurre y por qué?
5. Finaliza ambas sesiones ejecutando `COMMIT`;

Ejercicio 4 - Deadlock

Vamos a forzar un escenario donde la Transacción A espera a la B, y la Transacción B espera a la A.

1. **[Sesión A]:** Inicia transacción y descuenta stock del artículo 1.

```
BEGIN;  
UPDATE articulos SET stock = stock - 1 WHERE id_articulo = 1;
```

2. **[Sesión B]:** Inicia transacción y descuenta stock del artículo 2.

```
BEGIN;  
UPDATE articulos SET stock = stock - 1 WHERE id_articulo = 2;
```

3. **[Sesión A]:** Ahora el cliente de la Sesión A también quiere el artículo 2.

```
UPDATE articulos SET stock = stock - 1 WHERE id_articulo = 2;  
(Notarás que se queda esperando, ya que la Sesión B lo tiene bloqueado. Esto es normal).
```

4. **[Sesión B]:** Al mismo tiempo, el cliente de la Sesión B recuerda que también quería el artículo 1.

```
UPDATE articulos SET stock = stock - 1 WHERE id_articulo = 1;
```

5. **Responde:** ¿Qué mensaje de error exacto te arrojó PostgreSQL en alguna de las dos ventanas? ¿Qué acción tomó el SGBD para resolver el problema?

Ejercicio 5

```
BEGIN;
```

```
UPDATE billetera_clientes SET saldo = saldo - 500000 WHERE  
id_cliente = 2;
```

```
SELECT * FROM clientes;
```

1. **Responde:** ¿Qué mensaje de error obtuviste en el paso 2? ¿Por qué PostgreSQL no te deja ni siquiera ejecutar un simple `SELECT`?
Como vimos en la teoría, si una instrucción falla dentro de un `BEGIN...COMMIT`, PostgreSQL marca toda la transacción como abortada. El `CHECK` de saldo ≥ 0 generó la excepción que abortó la transacción.
2. **Escribe el comando necesario** para salir de este estado de error y liberar la consola.

El único comando posible es ROLLBACK, incluso si queremos comitear los cambios postgres ejecutará un rollback.

Ejercicio 6: El problema de la "Lectura No Repetible" (MVCC vs B2F)

Por defecto, PostgreSQL privilegia la concurrencia sobre el aislamiento estricto. Entonces, si no usamos bloqueos explícitos, podemos sufrir el problema de leer datos que cambian a mitad de nuestra transacción.

Paso 1: El Problema

[Sesión A]: Inicia y lee el precio del artículo 1.

```
BEGIN;
```

```
SELECT precio_unitario FROM articulos WHERE id_articulo = 1;
```

[Sesión B]: Inicia, modifica ese mismo precio y confirma.

```
BEGIN;
```

```
UPDATE articulos SET precio_unitario = precio_unitario + 500 WHERE  
id_articulo = 1;
```

```
COMMIT;
```

[Sesión A]: Vuelve a ejecutar el mismo `SELECT`.

Responde: ¿El valor cambió dentro de tu misma transacción? Cierra la transacción de A con `COMMIT`;

Paso 2: La Solución Pesimista (Protocolo B2F explícito)

Como vimos en la teoría, podemos evitar esto colocando un candado de lectura.

[Sesión A]: Inicia la transacción exigiendo un bloqueo compartido.

```
BEGIN;
```

```
SELECT precio_unitario FROM articulos WHERE id_articulo = 1 FOR  
SHARE;
```

[Sesión B]: Intenta modificar el precio.

```
BEGIN;
```

```
UPDATE articulos SET precio_unitario = precio_unitario + 500 WHERE  
id_articulo = 1;
```

Responde: ¿Qué sucede en B? ¿Se logró evitar la lectura no repetible obligando a B a esperar? (Cancela ambas operaciones ejecutando ROLLBACK en ambas pestañas).

Paso 3: La Solución Optimista (Nivel de Aislamiento MVCC)

Postgres ofrece una forma de garantizar lecturas repetibles sin bloquear a los demás, usando fotos (snapshots) de los datos.

[Sesión A]: Inicia la transacción con un nivel de aislamiento estricto.

```
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

```
SELECT precio_unitario FROM articulos WHERE id_articulo = 1;
```

[Sesión B]: Modifica el precio y confirma (¡Postgres lo permitirá porque no hay bloqueos!).

```
BEGIN;
```

```
UPDATE articulos SET precio_unitario = precio_unitario + 500 WHERE  
id_articulo = 1;
```

```
COMMIT;
```

[Sesión A]: Vuelve a ejecutar el `SELECT`.

Responde: ¿Qué valor ves ahora en A? ¿Se mantuvo la consistencia a pesar de que B ya cambió el dato real? Finaliza con `COMMIT`;

Paso 4:

[Sesión A]: Recordá que en tu "foto" (snapshot) el precio sigue siendo el viejo, pero en la realidad, la Sesión B ya lo subió \$500. Ahora intentaremos subirle el precio nosotros también basándonos en nuestra foto:

```
UPDATE articulos SET precio_unitario = precio_unitario + 200 WHERE  
id_articulo = 1;
```

Responde: ¿Qué mensaje de error exacto te arrojó PostgreSQL? ¿Por qué el motor tomó la decisión de abortar tu transacción en lugar de aplicar el UPDATE? Cierra la consola ejecutando ROLLBACK;.